



Logic Design

Ch 2 Part A

Boolean Algebra and Logic Gates

Instructor : Hasan Al Serhan

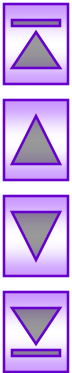
IT Department

Al Balqa Applied University



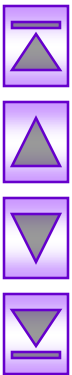
Introduction to Logic Gates

- Logic Gates
 - ❖ The Inverter
 - ❖ The AND Gate
 - ❖ The OR Gate
 - ❖ The NAND Gate
 - ❖ The NOR Gate
 - ❖ The XOR Gate
 - ❖ The XNOR Gate
- Drawing Logic Circuit
- Analysing Logic Circuit



Introduction to Logic Gates

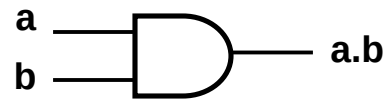
- Universal Gates: NAND and NOR
 - ❖ NAND Gate
 - ❖ NOR Gate
- Implementation using NAND Gates
- Implementation using NOR Gates
- Implementation of SOP Expressions
- Implementation of POS Expressions
- Positive and Negative Logic
- Integrated Circuit Logic Families



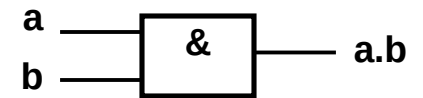
Logic Gates

■ Gate Symbols

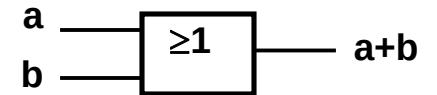
AND



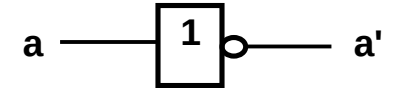
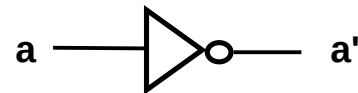
Symbol set 2
(ANSI/IEEE Standard 91-1984)



OR



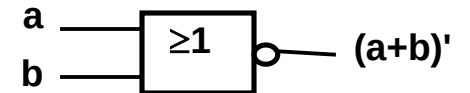
NOT



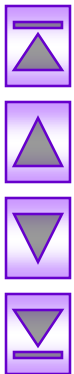
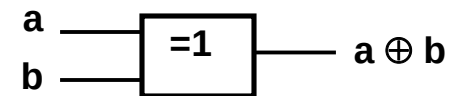
NAND



NOR



EXCLUSIVE OR



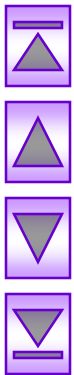
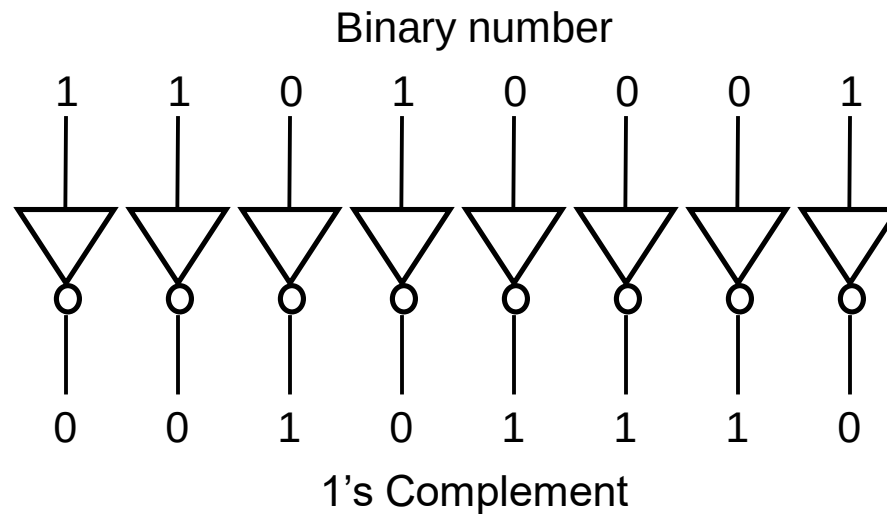
Logic Gates: The Inverter

- The **Inverter**



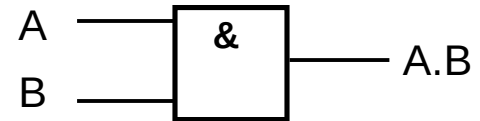
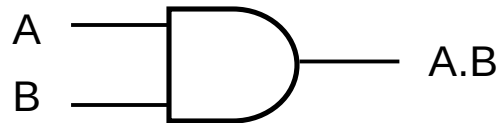
A	A'
0	1
1	0

- Application of the inverter: complement.

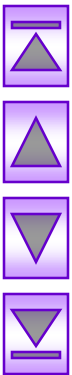


Logic Gates: The AND Gate

- The **AND** Gate

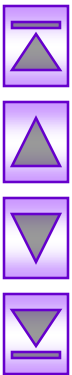
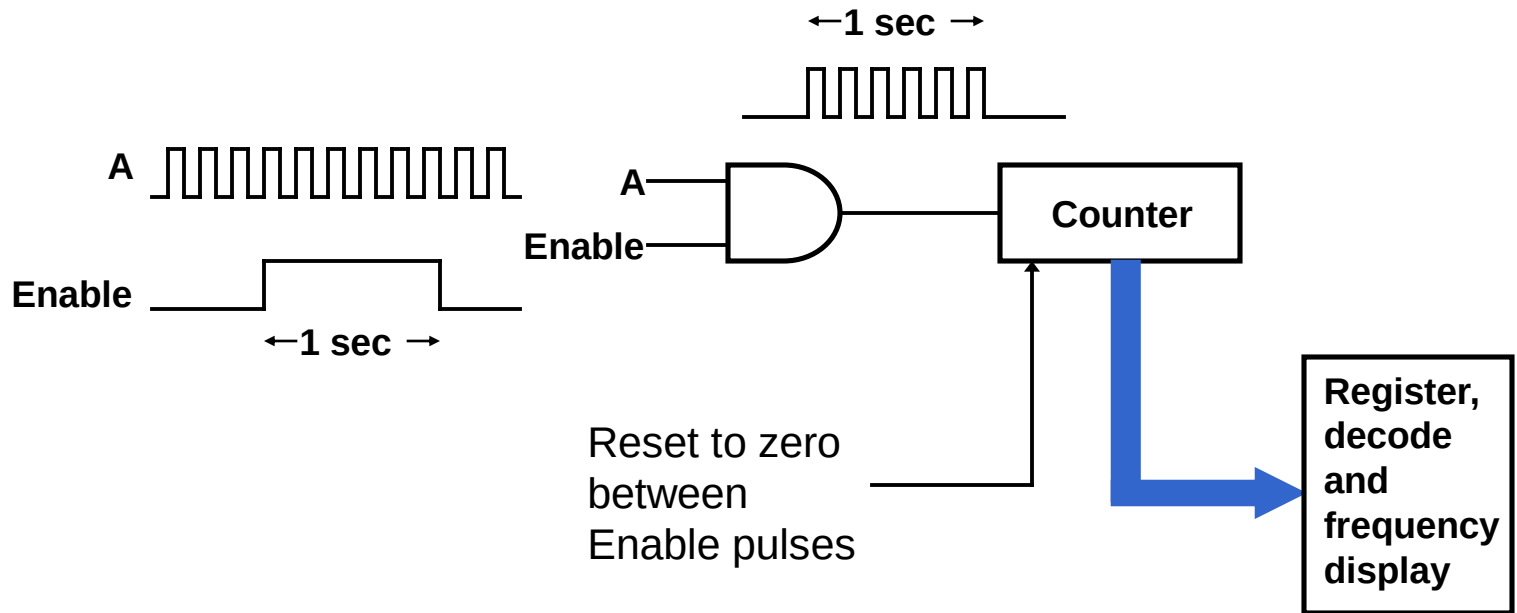


A	B	A . B
0	0	0
0	1	0
1	0	0
1	1	1



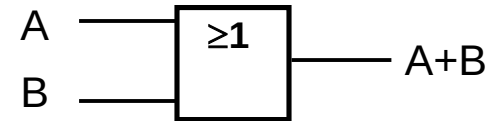
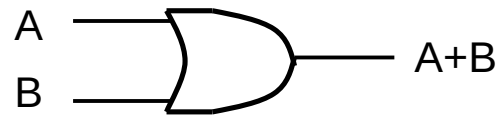
Logic Gates: The AND Gate

- Application of the AND Gate

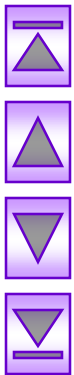


Logic Gates: The OR Gate

- The **OR** Gate

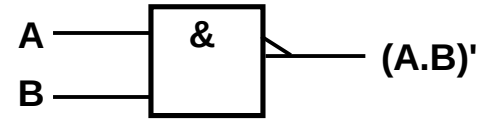
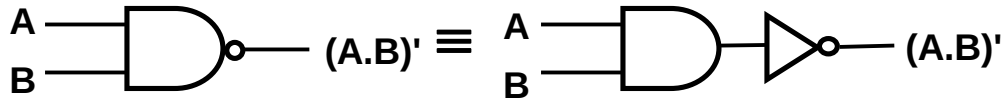


A	B	$A + B$
0	0	0
0	1	1
1	0	1
1	1	1

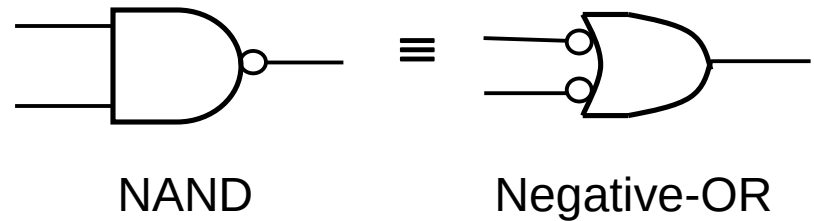


Logic Gates: The NAND Gate

- The **NAND** Gate

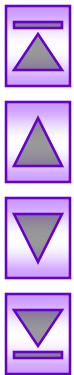


A	B	$(A.B)'$
0	0	1
0	1	1
1	0	1
1	1	0



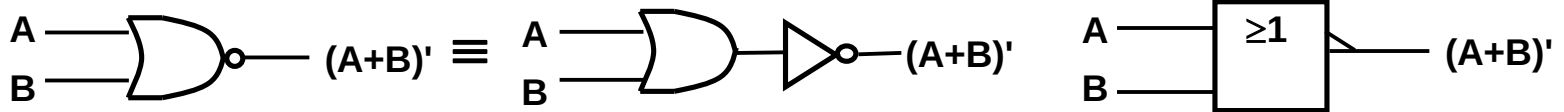
NAND

Negative-OR

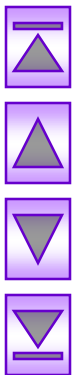
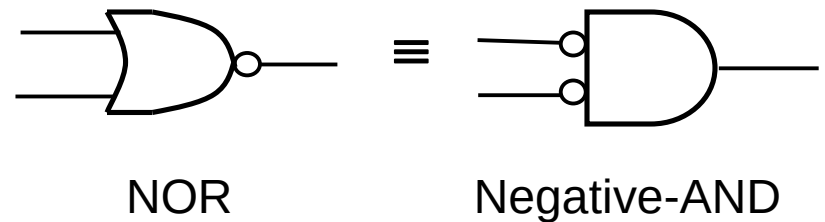


Logic Gates: The NOR Gate

- The **NOR** Gate

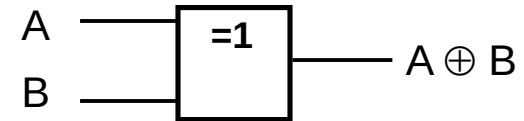
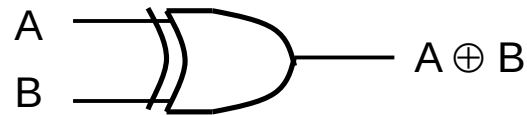


A	B	$(A+B)'$
0	0	1
0	1	0
1	0	0
1	1	0

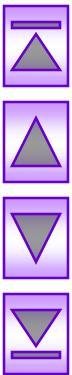


Logic Gates: The XOR Gate

- The **XOR** Gate

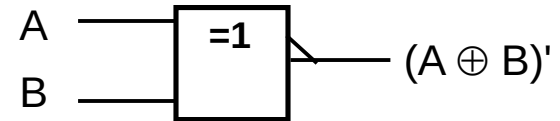
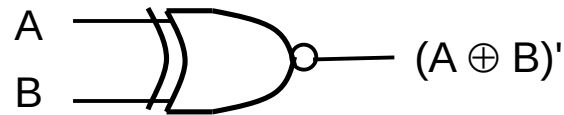


A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

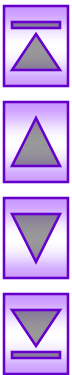


Logic Gates: The XNOR Gate

- The **XNOR** Gate

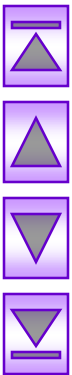
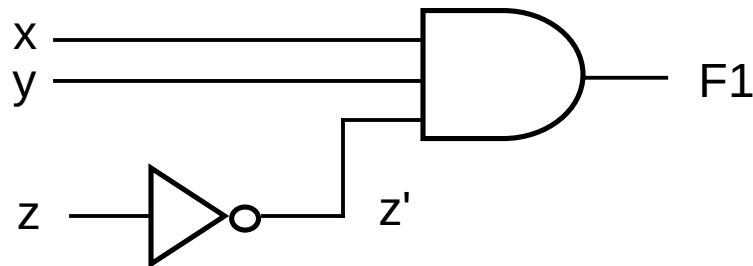


A	B	$(A \oplus B)'$
0	0	1
0	1	0
1	0	0
1	1	1



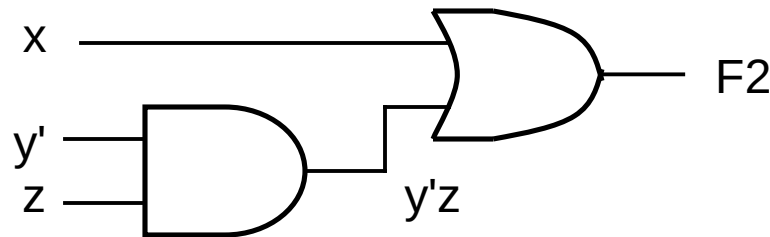
Drawing Logic Circuit

- When a Boolean expression is provided, we can easily draw the logic circuit.
- Examples:
 - (i) $F1 = xyz'$ (note the use of a 3-input AND gate)

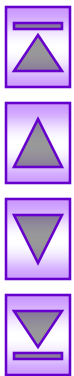
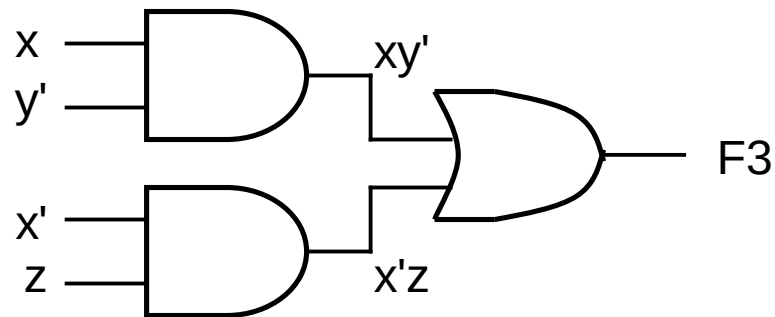


Drawing Logic Circuit

(ii) $F2 = x + y'z$ (can assume that variables and their complements are available)



(iii) $F3 = xy' + x'z$

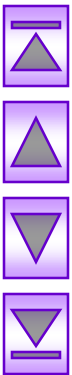


Solve it yourself (Exercise 4.1)

Q1. Draw a logic circuit for $BD + BE + D'F$

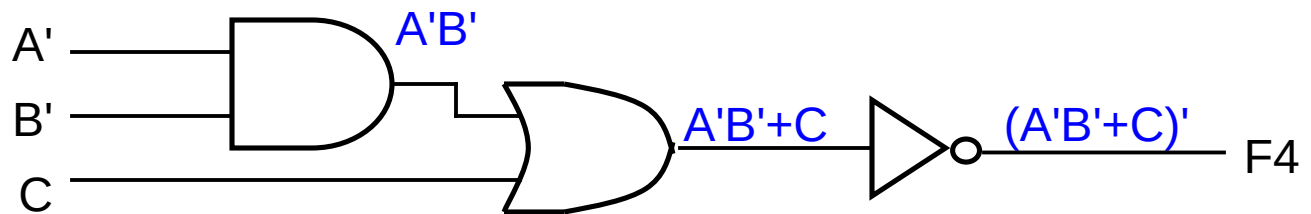
Q2. Draw a logic circuit for

$$A'BC + B'CD + BC'D + ABD'$$

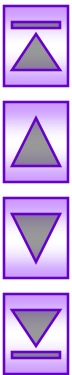


Analysing Logic Circuit

- When a logic circuit is provided, we can analyse the circuit to obtain the logic expression.
- Example: What is the Boolean expression of F4?

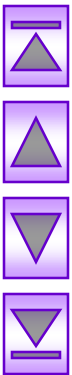
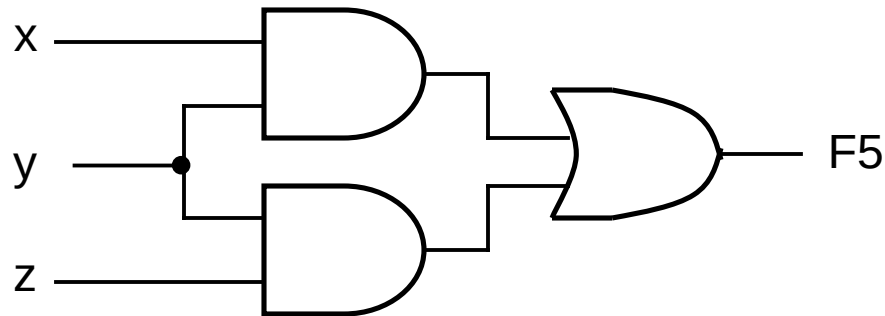


$$F4 = (A'B'+C)' = (A+B).C'$$



Solve it yourself (Exercise 4.2)

- What is Boolean expression of F5?

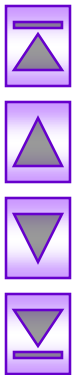


Universal Gates: NAND and NOR

- AND/OR/NOT gates are sufficient for building any Boolean functions.
- However, other gates are also used because:
 - (i) usefulness
 - (ii) economical on transistors
 - (iii) self-sufficient

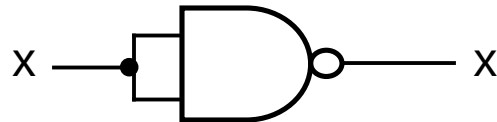
NAND/NOR: economical, self-sufficient

XOR: useful (e.g. parity bit generation)

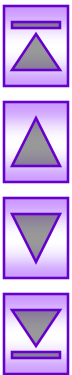


NAND Gate

- NAND gate is **self-sufficient** (can build any logic circuit with it).
- Can be used to implement AND/OR/NOT.
- Implementing an inverter using NAND gate:

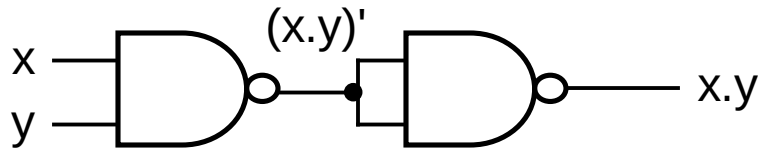


$$(x.x)' = x' \quad (\text{T1: idempotency})$$



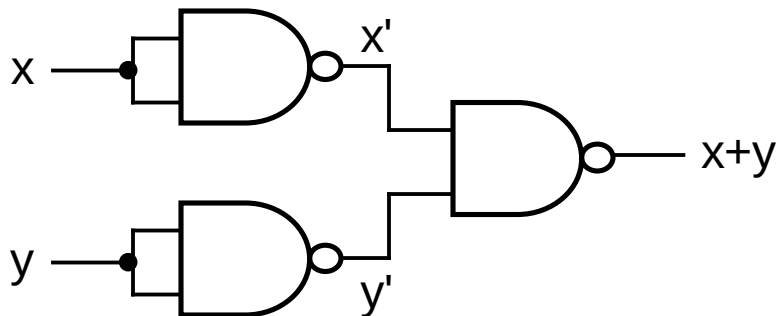
NAND Gate

- Implementing AND using NAND gates:

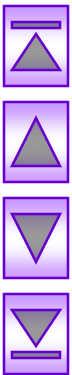


$$\begin{aligned} ((xy)'(xy)')' &= ((xy)')' && \text{idempotency} \\ &= (xy) && \text{involution} \end{aligned}$$

- Implementing OR using NAND gates:

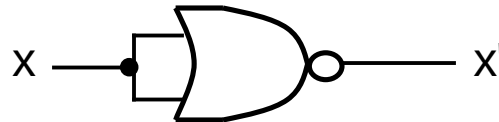


$$\begin{aligned} ((xx)'(yy)')' &= (x'y')' && \text{idempotency} \\ &= x''+y'' && \text{DeMorgan} \\ &= x+y && \text{involution} \end{aligned}$$

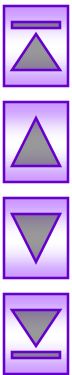


NOR Gate

- NOR gate is also self-sufficient.
- Can be used to implement AND/OR/NOT.
- Implementing an inverter using NOR gate:

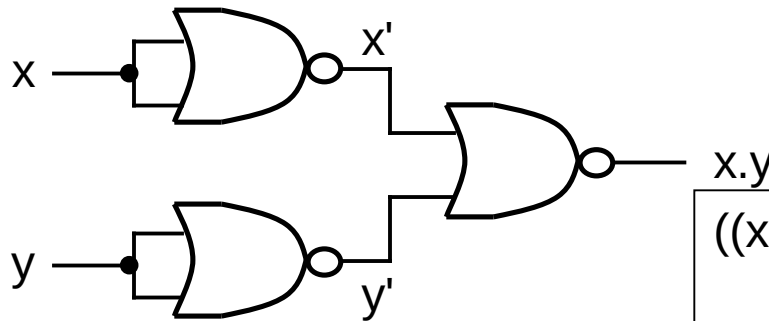


$$(x+x)' = x' \quad (\text{T1: idempotency})$$



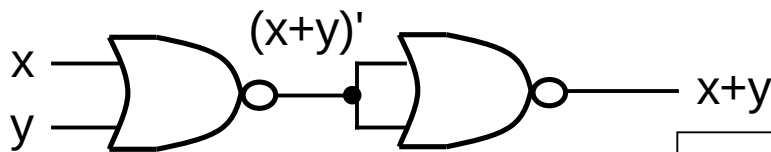
NOR Gate

- Implementing AND using NOR gates:

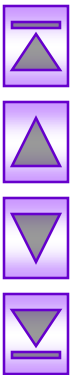


$$\begin{aligned} ((x+x)' + (y+y)')' &= (x' + y')' && \text{idempotency} \\ &= x'' \cdot y'' && \text{DeMorgan} \\ &= x \cdot y && \text{involution} \end{aligned}$$

- Implementing OR using NOR gates:



$$\begin{aligned} ((x+y)' + (x+y)')' &= ((x+y)')' && \text{idempotency} \\ &= (x+y) && \text{involution} \end{aligned}$$



Implementation using NAND gates

- Possible to implement any Boolean expression using NAND gates.

Procedure:

- (i) Obtain sum-of-products Boolean expression:

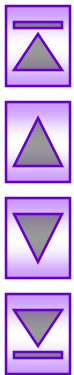
$$\text{e.g. } F3 = xy' + x'z$$

- (ii) Use DeMorgan theorem to obtain expression using 2-level NAND gates

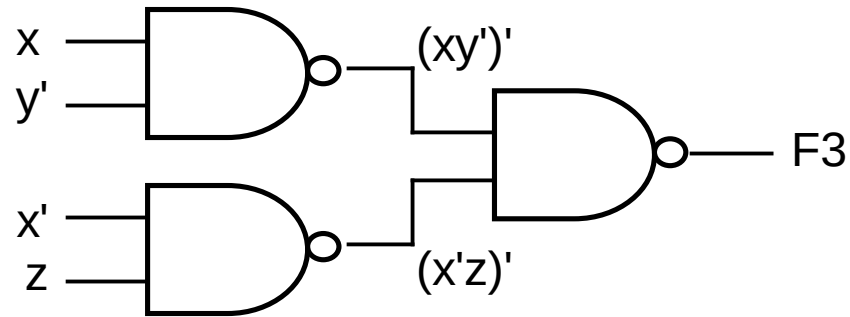
$$\text{e.g. } F3 = xy' + x'z$$

$$= (xy' + x'z)' \quad \text{involution}$$

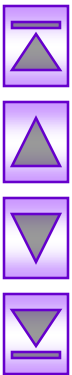
$$= ((xy')' \cdot (x'z)')' \quad \text{DeMorgan}$$



Implementation using NAND gates



$$F3 = ((xy')' \cdot (x'z)')' = xy' + x'z$$



Implementation using NOR gates

- Possible to implement boolean expression using NOR gates.

Procedure:

- Obtain product-of-sums Boolean expression:

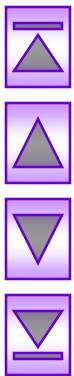
$$\text{e.g. } F6 = (x+y).(x'+z)$$

- Use DeMorgan theorem to obtain expression using 2-level NOR gates.

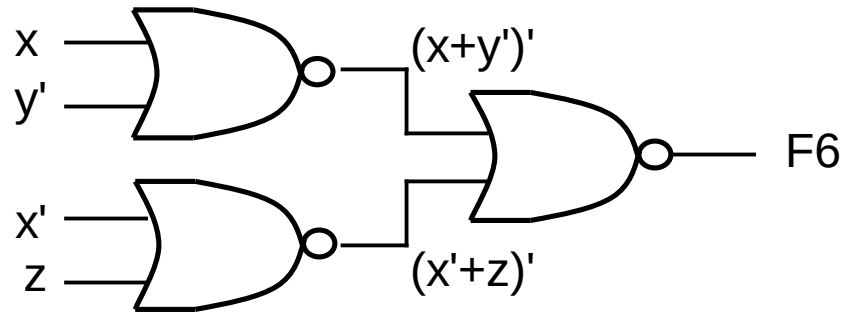
$$\text{e.g. } F6 = (x+y).(x'+z)$$

$$= ((x+y).(x'+z))' \quad \text{involution}$$

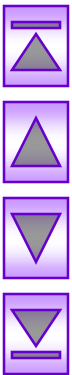
$$= ((x+y)' + (x'+z)')' \quad \text{DeMorgan}$$



Implementation using NOR gates

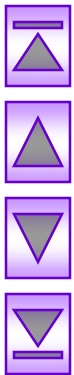
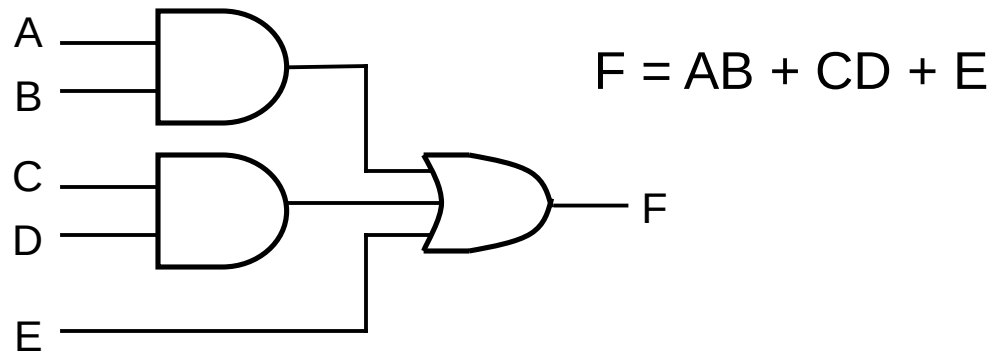


$$F6 = ((x+y')' + (x'+z)')' = (x+y').(x'+z)$$



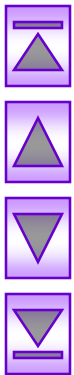
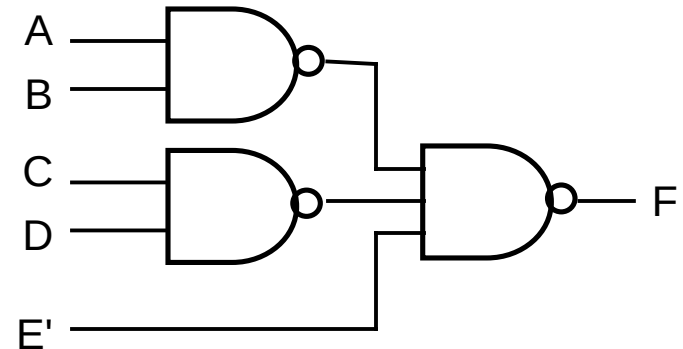
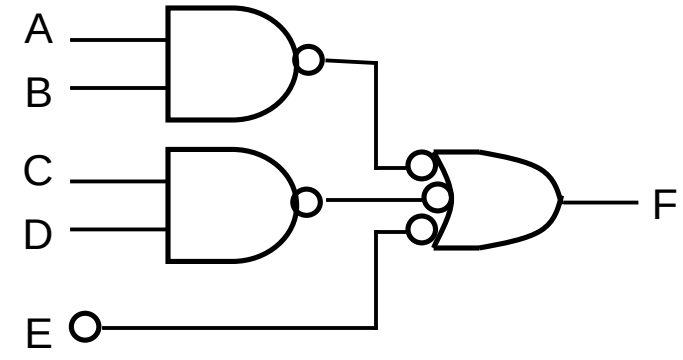
Implementation of SOP Expressions

- Sum-of-Products expressions can be implemented using:
 - ❖ 2-level AND-OR logic circuits
 - ❖ 2-level NAND logic circuits
- AND-OR logic circuit



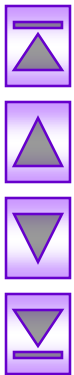
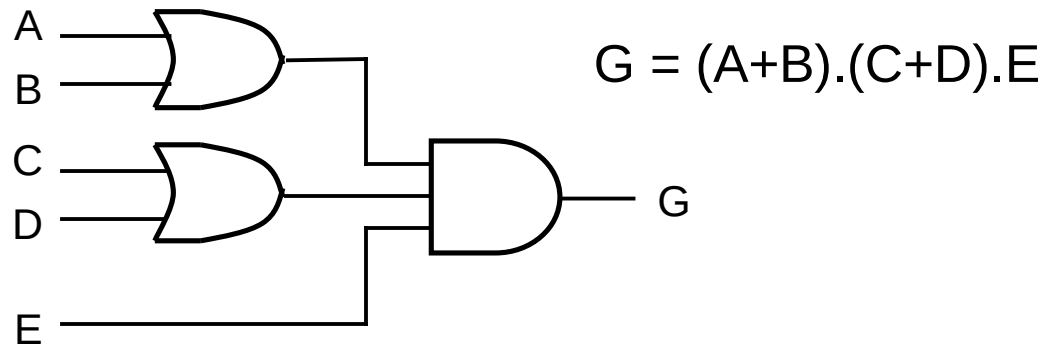
Implementation of SOP Expressions

- NAND-NAND circuit (by circuit transformation)
 - add double bubbles
 - change OR-with-inverted-inputs to NAND & bubbles at inputs to their complements



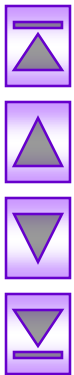
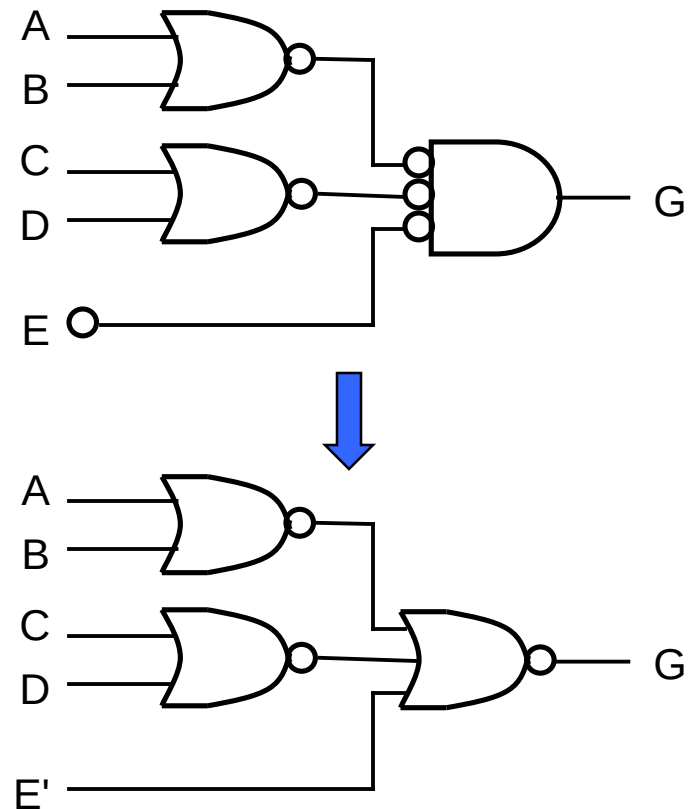
Implementation of POS Expressions

- Product-of-Sums expressions can be implemented using:
 - ❖ 2-level OR-AND logic circuits
 - ❖ 2-level NOR logic circuits
- OR-AND logic circuit



Implementation of POS Expressions

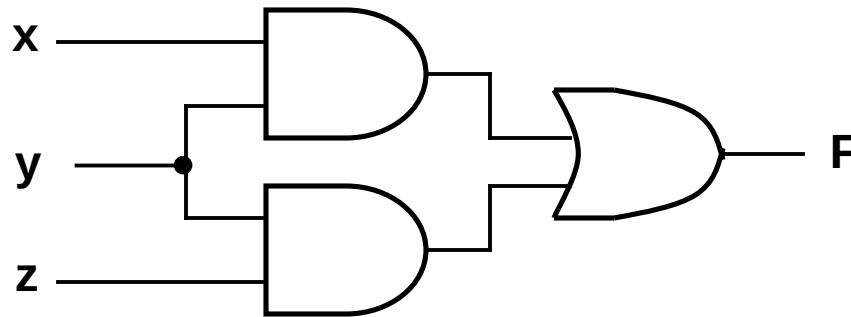
- NOR-NOR circuit (by circuit transformation):
 - a) add double bubbles
 - b) changed AND-with-inverted-inputs to NOR & bubbles at inputs to their complements



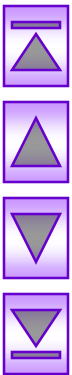
Solve it yourself (Exercise 4.3)

Q1. Draw a logic circuit for $BD + BE + D'F$ using only NAND gates. Use both DeMorgan method and SOP method.

Q2. Transform the following AND-OR Circuit to NAND circuit.

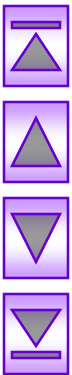


Q3. Using only NOR gates, draw a logic circuit using POS method for $(A+B+C')(B'+C'+D)$



Positive & Negative Logic

- In logic gates, usually:
 - ❖ H (high voltage, 5V) = 1
 - ❖ L (low voltage, 0V) = 0
- This convention – **positive logic**.
- However, the reverse convention, **negative logic** possible:
 - ❖ H (high voltage) = 0
 - ❖ L (low voltage) = 1
- Depending on convention, same gate may denote different Boolean function.



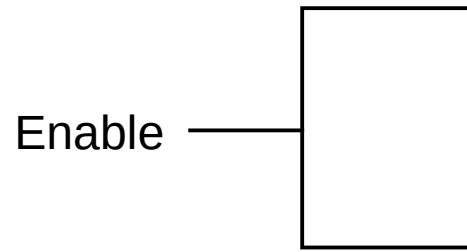
Positive & Negative Logic

- A signal that is set to logic 1 is said to be *asserted*, or *active*, or *true*.
- A signal that is set to logic 0 is said to be *deasserted*, or *negated*, or *false*.
- Active-high signal names are usually written in uncomplemented form.
- Active-low signal names are usually written in complemented form.



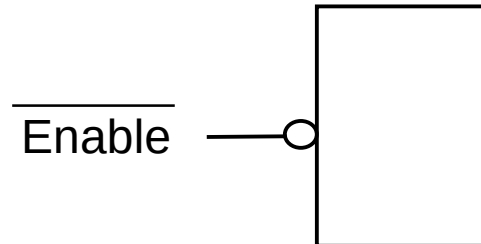
Positive & Negative Logic

Positive logic:

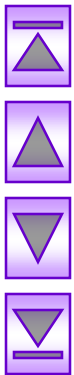


Active High:
0: Disabled
1: Enabled

Negative logic:

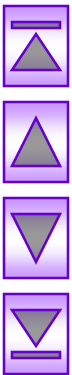


Active Low:
0: Enabled
1: Disabled



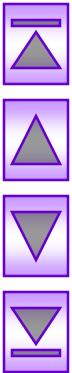
Integrated Circuit Logic Families

- Some digital integrated circuit families: TTL, CMOS, ECL.
- **TTL: Transistor-Transistor Logic.**
 - ❖ Uses bipolar junction transistors
 - ❖ Consists of a series of logic circuits: standard TTL, low-power TTL, Schottky TTL, low-power Schottky TTL, advanced Schottky TTL, etc.



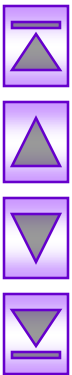
Integrated Circuit Logic Families

TTL Series	Prefix Designation	Example of Device
Standard TTL	54 or 74	7400 (quad NAND gates)
Low-power TTL	54L or 74L	74L00 (quad NAND gates)
Schottky TTL	54S or 74S	74S00 (quad NAND gates)
Low-power Schottky TTL	54LS or 74LS	74LS00 (quad NAND gates)



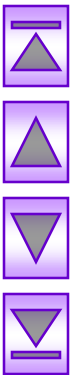
Integrated Circuit Logic Families

- **CMOS**: **C**omplementary **M**etal-**O**xide **S**emiconductor.
 - ❖ Uses field-effect transistors
- **ECL**: **E**mitter **C**oupled **L**ogic.
 - ❖ Uses bipolar circuit technology.
 - ❖ Has fastest switching speed but high power consumption.



Integrated Circuit Logic Families

- Performance characteristics
 - ❖ Propagation delay time.
 - ❖ Power dissipation.
 - ❖ **Fan-out**: Fan-out of a gate is the maximum number of inputs that the gate can drive.
 - ❖ Speed-power product (SPP): product of the propagation delay time and the power dissipation.



Summary

Logic Gates

AND,
OR,
NOT

NAND

NOR

Implementation of a
Boolean expression
using these
Universal gates.

Drawing Logic Circuit

Given a Boolean
expression, draw the
circuit.

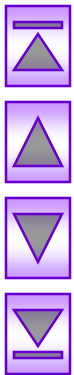
Implementation of SOP and POS Expressions

Concept of Minterm
and Maxterm

Analyzing Logic Circuit

Given a circuit, find
the function.

Positive and
Negative Logic



End of segment

